# Architecture de la bibliothèque

Guide de lecture : à quoi sert chaque dossier, à quoi sert chaque fichier

Documentation Architecturale

10 août 2026

## Table des matières


**1 Comment lire ce document** **2**

**2 Vue d'ensemble : la chaîne de traitement** **2**
2.1 Les trois endroits où quelque chose se décide . . . . . 2
2.2 Pourquoi séparer `model` de `rendering` ? . . . . . 2
2.3 Les trois règles qui reviennent partout . . . . . 3

**3 Le dossier racine : `umlg/`** **3**

**4 `annotation/` — le vocabulaire offert à l'utilisateur** **4**

**5 `model/` — le diagramme, dit et non dessiné** **6**

**6 `extraction/` — la lecture du code** **7**
6.1 `extraction/member/` — ce qui remplit une boîte . . . . . 8
6.2 `extraction/relation/` — ce qui tend un trait . . . . . 9
6.3 `extraction/rules/` — ce que le projet règle une fois pour toutes . . . . . 10

**7 `rendering/` — le dessin** **10**
7.1 `rendering/element/` — un morceau du dessin . . . . . 11
7.2 `rendering/syntax/` — ce que le moteur exige . . . . . 11

**8 `processing/` — la vérification à la compilation** **12**

**9 Ajouter une nature de lien : le mode d'emploi** **13**

**10 `src/test/` — les tests, en miroir** **14**

**11 Récapitulatif en une page** **14**

# 1 Comment lire ce document

Ce document répond à une seule question, fichier par fichier : *pourquoi celui-là existe-t-il, et que se passerait-il s'il n'existait pas ?* Il ne remplace pas les spécifications des annotations (`annotation.tex`), qui décrivent ce que l'utilisateur *écrit* ; il décrit ce que la bibliothèque *fait* de ce qu'il a écrit.

Le vocabulaire tient en trois mots, employés partout dans le code :

— une **entité** est une classe qui apparaîtra dans le diagramme (une boîte) ;
— un **lien** (ou *relation*) est un trait entre deux boîtes ;
— un **membre** est un attribut ou une méthode, écrit à l'intérieur d'une boîte.

# 2 Vue d'ensemble : la chaîne de traitement

La bibliothèque transforme du code Java annoté en une image. Ce trajet se fait en quatre étapes, et *chaque dossier correspond exactement à une étape*. C'est la clé de lecture principale : si vous savez à quelle étape appartient un problème, vous savez dans quel dossier chercher.

---

`annotation/` et le fichier `umlg.rules` ce que l'utilisateur écrit

↓

`extraction/` lecture du code compilé, par réflexion

↓

`model/` le diagramme, décrit sans savoir comment il sera dessiné

↓

`rendering/` traduction en code PlantUML, puis en image

---

Un cinquième dossier, `processing/`, est *hors* de cette chaîne : il s'exécute à l'intérieur du compilateur, et non à la génération. Il est décrit à part.

## 2.1 Les trois endroits où quelque chose se décide

Cette question revient sans cesse : *où écrit-on ce réglage ?* La réponse tient en une phrase, et elle explique la présence du fichier de règles à côté des annotations.

— Si la décision **désigne un élément par son nom** — cette classe-ci s'appelle ainsi, cet attribut-là est un lien — c'est une **annotation**.
— Si elle vaut pour **tout le projet, ou pour une vue**, c'est une règle du fichier `umlg.rules`.
— Si elle ne dit **rien du diagramme** — son habillage, le sort de la génération précédente — c'est une **option d'appel** du générateur.

D'où une règle de priorité sans exception : **l'annotation l'emporte toujours sur la règle**. Le cas particulier, écrit sur l'élément, bat le réglage qui vaut partout.

## 2.2 Pourquoi séparer `model` de `rendering` ?

C'est la séparation la plus importante, et la moins évidente. On pourrait croire qu'un modèle intermédiaire est une complication inutile : pourquoi ne pas écrire directement le code PlantUML pendant qu'on lit les annotations ?

Parce que les deux étapes n'ont pas les mêmes questions à trancher. L'extraction se demande *« que dit ce code ? »* ; le rendu se demande *« comment dessiner cela sans casser la syntaxe de PlantUML ? »*. Mélangées, ces deux questions se contaminent : on se met à refuser un nom de classe parfaitement légitime en Java — `"Compte (bancaire)"` — au motif qu'il gênerait le moteur de rendu. Le défaut serait alors placé au mauvais endroit, et l'utilisateur paierait pour une contrainte qui ne le regarde pas.

Séparés, chacun fait son travail : l'extraction accepte tout texte qui a un sens, et le rendu l'*encode* au moment de l'écrire, en fonction de l'endroit où il atterrit. C'est le même raisonnement qu'entre « filtrer les apostrophes » et « utiliser une requête préparée ».

Bénéfice secondaire, mais réel : le jour où un autre moteur est visé (Mermaid, Graphviz), seul `rendering/` est à remplacer.

### 2.3 Les trois règles qui reviennent partout

Une fois ces trois règles comprises, la plupart des décisions du code s'expliquent d'elles-mêmes.

**1. Les gardes appartiennent au modèle, le contexte appartient à l'extraction.** Une règle du type « une multiplicité s'écrit `0..*` » est vérifiée dans `model/`, parce que le modèle doit rester sain quelle que soit la façon dont on le construit. Mais le modèle ne connaît pas votre code : il peut seulement dire *« `"0..x"` n'est pas une multiplicité valide »*. Sur un projet de quarante classes, ce message ne mène nulle part. L'extraction, elle, sait *où* : elle rattrape l'erreur et y ajoute `com.banque.Compte.solde`.

**2. La sûreté syntaxique s'obtient à la sortie, jamais à l'entrée.** Aucun caractère n'est interdit dans un nom, un rôle ou une étiquette. Ce sont des textes destinés à être lus par un humain. C'est `PlantUmlEscaper` qui les rend inoffensifs au moment de les écrire. Seul ce dont le vocabulaire est *fermé* — les multiplicités, les couleurs — est vérifié par liste blanche.

**3. L'échec est franc, parce que PlantUML ne l'est pas.** Face à un nom de classe inconnu, PlantUML n'échoue pas : il invente une boîte vide portant ce nom. Face à une couleur inconnue, il produit une image d'erreur parfaitement valide. Dans les deux cas, le résultat est un fichier de taille normale, indiscernable d'un vrai diagramme. C'est pourquoi la bibliothèque vérifie elle-même, en amont, et refuse bruyamment.

## 3 Le dossier racine : `umlg/`

Peu de fichiers, et c'est voulu : c'est la porte d'entrée, et tout ce qui s'y trouve est public.

| Fichier | Rôle |
|---|---|
| `UMLGenerator` | La façade publique. C'est la seule classe qu'un utilisateur de la bibliothèque a besoin de connaître. Elle enchaîne les deux grandes étapes — scanner, puis écrire — et rien d'autre. Elle se construit par un *builder* (`sourceDirectory`, puis `outputFile` pour un seul dessin ou `outputDirectory` pour toutes les vues), ce qui permet de vérifier tôt que le dossier existe et que le format demandé est bien du `.png` ou du `.svg`. C'est aussi là que se posent les options d'appel — `style`, `keepPrevious` —, celles qui ne disent rien du diagramme. Contient un `main`, pour la ligne de commande. |
| `UmlStyle` | L'habillage : couleurs, épaisseur des traits, police. Ne change jamais ce que le diagramme *dit* — les mêmes boîtes, les mêmes traits — seulement son apparence. |
| `RapportDeGeneration` | Ce que la génération a fait : combien de diagrammes, où, combien de classes dessinées et ignorées. La bibliothèque le *rend* au lieu de l'afficher : choisir le canal n'appartient pas à une bibliothèque, mais à celui qui l'appelle. |
| `Avertissements` | Où partent les avertissements, pour la même raison. Le plugin Maven les dirige vers son *log*, la ligne de commande vers la sortie d'erreur. |

# 4 `annotation/` — le vocabulaire offert à l'utilisateur

Ce dossier est, avec `UMLGenerator`, la **surface publique** de la bibliothèque : tout le reste est de la mécanique interne. Les fichiers y sont très courts, car une annotation Java ne fait rien — elle se contente de *déclarer*. Aucune validation n'y a lieu : le compilateur garantit les types, le modèle garantit le reste.

| Fichier | Rôle |
|---|---|
| `UMLClass` | Le déclencheur. Une classe sans cette annotation n'existe pas pour la bibliothèque. Porte les quatre réglages de la boîte : `name` (l'étiquette affichée), `color`, `stereotype`, et `strategy` (le seuil de visibilité, voir ci-dessous). |
| `UMLVisibility` | L'énumération des seuils : `NONE` (aucun membre affiché, valeur par défaut), `PUBLIC`, `PROTECTED`, `PACKAGE`, `PRIVATE` (tout). Répond à la question « jusqu'à quel niveau de visibilité Java les membres apparaissent-ils ? ». |

| Fichier | Rôle |
|---|---|
| `UMLInclude`<br>`UMLExclude` | Les deux dérogations, posées sur un attribut ou une méthode. Elles l'emportent toujours sur la stratégie : `@UMLInclude` fait apparaître ce qu'elle masquerait, `@UMLExclude` masque ce qu'elle afficherait. En cas de conflit, `@UMLExclude` gagne. |
| `UMLAssociation` | Posée sur un *attribut*, elle le transforme en trait entre deux boîtes. Tous ses paramètres sont facultatifs : cardinalités, rôles, étiquette, sens du trait, et `target` pour désigner explicitement la classe visée quand elle ne peut pas se déduire. |
| `UMLRelationType` | La sémantique du trait : `ASSOCIATION` (lien simple), `AGGREGATION` (losange creux), `COMPOSITION` (losange plein). Le code Java ne distingue pas ces trois cas — `private List<Ligne> lignes` peut être l'un ou l'autre — seul le métier tranche. |
| `UMLDirection` | Une contrainte de *placement* : `UP`, `DOWN`, `LEFT`, `RIGHT`, ou `NONE`. Ne change rien au sens du diagramme, seulement à sa mise en page. |
| `UMLDependency` | Posée sur la *classe*, elle déclare « celle-ci a besoin de celle-là » sans la détenir. C'est la seule façon de faire apparaître un lien que le code ne porte pas structurellement : un service appelé dans le corps d'une méthode n'est pas un attribut, rien ne pourrait en être déduit. |
| `UMLAssociationClass` | Une classe qui qualifie un lien — ce que Java ne sait pas exprimer autrement que par une classe intermédiaire. UML la relie au trait par un pointillé. |
| `UMLMultiplicity`<br>`UMLDerived`<br>`UMLNotation` | Trois précisions sur une ligne de boîte : combien d'éléments (`0..*`), si la propriété est *calculée* plutôt que stockée (la barre oblique d'UML), et sous quelle forme un lien se dessine — un trait, ou une ligne dans la boîte. |
| `UMLNote`<br>`UMLConstraint` | Ce qu'on ajoute au dessin sans l'inventer : une bulle de commentaire, et une règle formelle entre accolades (`{ordered}`). Toutes deux se posent sur une classe comme sur un membre. |
| `UMLPackage` | Le rectangle nommé dans lequel une boîte se range, avec sa couleur. C'est le *seul* moyen de grouper : ranger désigne des classes par leur nom, donc c'est une annotation, jamais une règle. |
| `UMLFlow` | Le sens dans lequel un dessin se déplie : de haut en bas, ou de gauche à droite. |

| Fichier | Rôle |
|---|---|
| `UMLDependencies`<br>`UMLNotes`<br>`UMLConstraints` | Purement techniques, jamais écrites à la main. Java exige un conteneur pour qu'une annotation puisse être *répétée* sur le même élément. C'est ce qui permet d'écrire plusieurs `@UMLDependency` d'affilée. |
| `package-info` | Documentation du paquetage : son statut public, et la marche à suivre pour l'étendre. |

# 5 `model/` — le diagramme, dit et non dessiné

Ce dossier décrit le diagramme *sans savoir* qu'il sera rendu par PlantUML. Tous les fichiers y sont des `record` Java : des objets immuables, qui vérifient leur cohérence à la construction et ne peuvent plus changer ensuite. Un modèle qui se laisserait modifier après coup ne pourrait rien garantir.

| Fichier | Rôle |
|---|---|
| `UmlProject` | L'ensemble des entités du diagramme, et les vues qu'on en tire. Refuse d'être vide : scanner un projet où personne n'a annoté doit produire une erreur compréhensible, pas une image blanche. |
| `UmlView` | Un sous-ensemble du diagramme, dessiné dans son propre fichier. Un projet réel ne tient pas sur une image : la vue intégrale sert de plan, et chaque vue détaille un sujet. |
| `UmlDiagram` | Les réglages qui valent pour le dessin d'ensemble : son titre, son sens de lecture, le sort des compartiments restés vides. |
| `UmlEntity` | Une boîte. Porte deux noms qu'il ne faut surtout pas confondre : `qualifiedName` est son *identité* (`com.banque.Compte`), `name` son *étiquette* (ce que le lecteur voit). L'étiquette est libre, donc ni unique ni reliée au code ; c'est pourquoi les liens désignent toujours leur cible par le nom qualifié, qui, lui, ne ment pas. Contient aussi ses membres et *tous* ses liens sortants. |
| `UmlField`<br>`UmlMethod` | Une ligne de la boîte : nom, type, symbole de visibilité (`+ - # ~`), caractère statique, et pour la méthode la liste des types de paramètres. |

| Fichier | Rôle |
|---|---|
| `UmlRelation` | **La pièce centrale pour la suite du projet.** C'est une interface *scellée* : elle énumère elle-même les natures de liens qui existent. Grâce à elle, une entité n'a qu'*une seule* liste de liens, toutes natures confondues. Voir la section 9, qui explique ce que cela change concrètement. |
| `UmlAssociation` | Un lien *structurel* : une extrémité détient l'autre. Porte les cardinalités, les rôles, le qualificatif, l'étiquette et le sens. Vérifie ici que les multiplicités sont bien formées *et* cohérentes : `5..2` est syntaxiquement irréprochable et vide de sens, PlantUML l'afficherait sans broncher. |
| `UmlDependency` | Un lien d'*usage* : la source a besoin de la cible, sans la détenir. D'où l'absence de cardinalité et de rôle — il n'y a pas d'extrémité à décrire, seulement un usage à signaler. |
| `UmlInheritance`<br>`UmlAssociationClass`<br>`UmlLayout` | Les trois autres natures de liens : l'héritage (lu dans `extends` et `implements`, sans rien annoter), le pointillé qui rattache une classe d'association à son trait, et le lien *invisible* qui ne sert qu'à placer les boîtes que rien ne relie. |
| `UmlNote`<br>`UmlPackage`<br>`UmlParameter` | Les petites pièces : une bulle de commentaire, un rectangle de regroupement, un paramètre de méthode. |
| `package-info` | Documentation du paquetage : où sont les gardes, et pourquoi tout y est immuable. |

**Une remarque sur l'absence de « source » dans les liens.** Ni `UmlAssociation` ni `UmlDependency` ne mentionnent la classe d'où part le trait. Ce n'est pas un oubli : la source *est* l'entité dans la liste de laquelle le lien figure. Un lien ne peut donc être ni orphelin, ni attribué à une source autre que celle qui le déclare — deux erreurs qu'il serait autrement possible de commettre.

# 6 `extraction/` — la lecture du code

Ce dossier lit les fichiers `.class` compilés et les traduit en modèle, par *réflexion* (l'API Java qui permet d'interroger une classe à l'exécution).

C'est le dossier le plus fourni, et il est découpé en **trois sous-dossiers**, parce que chacun de ces sujets grossit de son côté : ce qui remplit une boîte (`member/`), ce qui tend un trait entre deux boîtes (`relation/`), et ce que le projet a réglé une fois pour toutes (`rules/`). Réunis, ils se gênaient.

Seuls `ProjectScanner` et `EntityExtractor` sont publics ; presque tout le reste est volontairement invisible hors du paquetage. C'est là que se lit le contrat : ce qui n'est pas public n'a pas à rester stable.

| Fichier | Rôle |
|---|---|
| `ProjectScanner` | Parcourt un dossier de `.class`, les charge, et retient ceux qui portent `@UMLClass`. C'est aussi lui qui *signale* ce qui vaut d'être vu sans arrêter le dessin : boîtes vides faute de stratégie déclarée, compilation sans `-parameters`, règles restées lettre morte. |
| `EntityExtractor` | Le chef d'orchestre, et rien d'autre. Il valide l'en-tête, demande les membres, puis parcourt son **catalogue d'extracteurs de liens**. Il ne connaît aucune nature de lien en particulier — seulement leur liste. |
| `ClassHeaderValidator` | Les gardes sur ce que `@UMLClass` déclare : `name`, `stereotype`, `color`. Illustre la règle n°2 : le nom n'a qu'une contrainte (désigner quelque chose), la couleur est vérifiée par liste blanche. |
| `ConstructeurDeVues` | Taille le projet en vues : l'intégrale, celles que le fichier déclare, celles qu'il fait calculer (une par paquetage). Les vues sont découpées ici et non au rendu, car elles dépendent des règles du projet, que seule l'extraction a lues. |
| `TypeAffiche` | Écrit un type tel qu'il paraîtra dans une boîte, paramètres de type compris — et c'est ici que les renommages du fichier de règles (`ImmutableList` affiché « List ») s'appliquent. |
| `MembresOrdonnes`<br>`OrdreDuFichierClasse` | L'ordre des membres dans une boîte : l'alphabet par défaut, ou l'ordre du code si le projet le demande. La réflexion ne promet aucun ordre ; le second relit donc le fichier `.class` pour retrouver celui de la déclaration. |
| `PackageExtractor`<br>`NoteExtractor`<br>`ConstraintExtractor` | Les trois lectures secondaires : le rectangle de regroupement, les bulles de commentaire, les règles formelles entre accolades. |
| `package-info` | Documentation du paquetage : statut de chaque classe, et le point d'extension. |

## 6.1 `extraction/member/` — ce qui remplit une boîte

| Fichier | Rôle |
|---|---|
| `MemberExtractor` | Parcourt attributs et méthodes, et traduit ceux qui sont retenus. Écarte au passage un attribut promu en lien : en UML, une propriété se note dans la boîte *ou* comme un trait, jamais les deux. |

| Fichier | Rôle |
|---|---|
| `VisibilityPolicy` | Répond à « ce membre paraît-il ? », et « sous quel symbole ? ». Applique d'abord les dérogations, puis le seuil. Écrit avec un `switch` exhaustif : ajouter une valeur à `UMLVisibility` sans dire ce qu'elle retient devient une erreur de compilation. |
| `NoiseFilter` | *Reconnaît* la plomberie — getters, setters, `toString`, accesseurs de `record`, méthodes que `javac` ajoute aux `enum` — sans jamais décider de la masquer : c'est le fichier de règles qui le demande, famille par famille. Détail crucial : la détection exige la convention JavaBean *complète* (préfixe suivi d'une majuscule, arité, type de retour). Un simple test de préfixe faisait disparaître silencieusement `setup()`, `issue()`, `getaway()`, `isoler()` — de vraies méthodes métier. |

## 6.2 `extraction/relation/` — ce qui tend un trait

| Fichier | Rôle |
|---|---|
| `RelationExtractor` | **Le point d'extension.** Une interface : « voici une classe, donne-moi les liens que tu sais y lire ». Une implémentation par nature de lien. |
| `AssociationExtractor`<br>`DependencyExtractor`<br>`InheritanceExtractor`<br>`AssociationClassExtractor` | Les quatre implémentations. Notez qu'un lien n'est *pas* soumis à la stratégie de visibilité : une composition ne disparaît pas parce que le champ qui la porte est privé — c'est même le cas le plus courant. L'héritage, lui, ne s'annote pas : il se lit dans `extends` et `implements`. |
| `TargetResolver`<br>`CibleDeduite` | La question la plus difficile du dossier : *vers quoi pointe ce lien quand l'annotation ne le dit pas ?* Savent déduire l'élément d'un tableau, le contenu d'une collection ou d'un `Optional`, la valeur d'une `Map`. Refusent franchement quand le code ne dit rien (`List` brute, `List<T>`) : deviner produirait un lien faux, qui se lit exactement comme un lien vrai. |
| `Emplacement` | Une seule méthode, et pourtant un fichier à elle : elle produit le « où » des messages d'erreur (`com.banque.Compte.solde`). Chaque extracteur à venir doit désigner la déclaration fautive de la même façon, et non chacun à la sienne. |

### 6.3 `extraction/rules/` — ce que le projet règle une fois pour toutes

Une annotation dit une chose sur un élément. Écrire « affiche `ImmutableList` comme `List` » sur chacun des deux cents attributs d'un projet est un travail que personne ne fait, et que personne ne tient à jour. D'où un fichier, `umlg.rules`, posé à la racine du répertoire analysé — facultatif, et lu une seule fois avant la première classe.

| Fichier | Rôle |
|---|---|
| `LecteurDeRegles` | Lit le fichier ligne à ligne. Une règle commence par un mot-clé qui dit sa nature — il y en a onze — et c'est lui qui décide comment lire le reste. Le vocabulaire est **fermé** : un mot inconnu *arrête* la génération, plutôt que d'appliquer en silence des règles dont une partie serait ignorée. Chaque refus nomme le fichier et le numéro de ligne : il n'y a pas de compilateur pour relire ce que l'utilisateur écrit ici. |
| `Regles` | Le résultat de cette lecture, interrogé par tout le reste de l'extraction. Retient au passage quelles lignes ont *servi* : une règle qui ne vise plus rien — une classe renommée, une faute de frappe — est un no-op parfait, que rien d'autre ne signalerait. |
| `Motif`<br>`Bruit` | Les deux façons de désigner des membres : un motif de noms (`get*`, `*Cache`), et les quatre familles reconnues à leur *forme*. Un motif ne peut pas contenir de point : désigner un membre par son nom est le travail d'une annotation. |
| `DefinitionDeVue`<br>`Selecteur` | Ce qu'une vue retient, et comment on le désigne : une classe, un paquetage entier, ou le voisinage immédiat d'une classe. |

## 7 `rendering/` — le dessin

Ce dossier est le seul à connaître la syntaxe de PlantUML. Seul `PlantUmlWriter` est public. Il se découpe en deux sous-dossiers : `element/` écrit *un* morceau du dessin — une boîte, un trait —, et `syntax/` règle ce que le moteur exige de tous.

| Fichier | Rôle |
|---|---|
| `PlantUmlWriter` | Le chef d'orchestre du rendu : construit le document, puis demande à PlantUML de produire le `.svg` ou le `.png`. Applique la règle des **deux passes** — toutes les boîtes d'abord, tous les traits ensuite — car un trait nomme ses extrémités par un identifiant qui n'existe qu'une fois la boîte déclarée. |

| Fichier | Rôle |
|---|---|
| `PlantUmlSyntaxBuilder` | Assemble le document. Ne connaît plus de la syntaxe que l'*ordre* des morceaux, ce qu'aucun de ses collaborateurs ne peut savoir seul : les classes avant les liens, et le bloc de style en tête bien qu'il ne soit calculable qu'à la fin. |
| `AssociationsReciproques` | Deux attributs qui se font face ne sont qu'*une* association. Sans cette passe, le même lien serait dessiné deux fois, une fois depuis chaque bout. |
| `Proportions` | Mesure l'image produite et signale celle qu'on ne pourra pas lire — un ruban de douze contre un. C'est le seul contrôle qui exige que le dessin existe déjà. |
| `package-info` | Documentation du paquetage, dont l'asymétrie « ouvert à l'extraction, fermé au rendu ». |

## 7.1 `rendering/element/` — un morceau du dessin

| Fichier | Rôle |
|---|---|
| `ClassBoxRenderer` | Écrit une boîte : son en-tête, ses attributs, le filet séparateur, ses méthodes. L'ordre des morceaux de l'en-tête n'est pas libre — le moteur accepte `[[lien]] #couleur` et refuse l'inverse, sans le dire. |
| `RelationRenderer` `Extremite` | Écrivent un trait, quelle que soit sa nature, par un `switch` sur `UmlRelation`, et l'étiquette posée à chacun de ses bouts (« achats 0..* »). Retiennent au passage la largeur de la plus large : c'est ici, et nulle part ailleurs, que le diagramme apprend de quel espace il aura besoin entre ses boîtes. |
| `NoteRenderer` `PackageRenderer` | La bulle de commentaire, et le rectangle qui entoure un groupe de boîtes. |

## 7.2 `rendering/syntax/` — ce que le moteur exige

| Fichier | Rôle |
|---|---|
| `AliasRegistry` | L'annuaire du document : à chaque classe, l'identifiant court par lequel PlantUML la désignera. Traite deux pièges que PlantUML ne signale pas : un nom sans aucun caractère alphanumérique ASCII (un nom en idéogrammes, ou « ... ») donnerait un identifiant vide, et deux noms distincts peuvent se réduire au même (« Mon.Client » et « MonClient »), auquel cas PlantUML fusionnerait silencieusement les deux classes. |
| `PlantUmlEscaper` | L'encodage de sortie (règle n°2). Neutralise ce qui casserait la syntaxe ou permettrait une injection. Ce n'est pas cosmétique : les étiquettes PlantUML acceptent des balises, dont `<img:/chemin>`, qui fait lire un fichier local au moteur de rendu. |
| `ColorCodec` | Vérifie qu'une couleur existe vraiment, en interrogeant le catalogue de PlantUML — la seule autorité en la matière, et la raison pour laquelle cette vérification n'est pas à l'extraction. |
| `PlantUmlLayout` | Tout ce qui ne change rien à ce que le diagramme *dit*, et tout à la facilité avec laquelle on le *lit* : police, couleurs, coins arrondis, et surtout l'espacement entre boîtes, calculé d'après la plus longue étiquette — c'est ce réglage qui décide, à lui seul, si les textes se chevauchent. |
| `package-info` | Documentation du paquetage : ce qui relève du moteur, et ce qui n'en relève pas. |

# 8 `processing/` — la vérification à la compilation

Ce dossier est le seul à ne pas participer à la chaîne. Il contient un **processeur d'annotations** : du code qui s'exécute *à l'intérieur de `javac`*. Il ne produit rien ; il lit les annotations et refuse la compilation quand elles sont fautives.

Pourquoi le faire deux fois, puisque la génération vérifie déjà ? À cause du **moment**. Une annotation contradictoire découverte à la génération se paie d'un aller-retour : on lance l'outil, on lit un message, on retourne au code. Découverte à la compilation, elle est signalée comme n'importe quelle erreur Java — soulignée dans l'éditeur, à la ligne près, avant même d'avoir enregistré. C'est la différence entre un correcteur orthographique et une relecture.

| Fichier | Rôle |
|---|---|
| `UmlgProcessor` | Le point d'entrée, déclaré à `javac`. Il ne vérifie rien lui-même : il distribue chaque élément annoté aux vérificateurs. |

| Fichier | Rôle |
|---|---|
| Les `Verificateur*` | Un par sujet — les classes, les membres, les associations, les multiplicités, les contraintes, les notes… Chacun connaît une famille de fautes et le message qui l'explique. |
| `Rapporteur` | La distinction qui structure tout le dossier : une **erreur** arrête la compilation, un **avertissement** la laisse passer. La frontière est une règle, pas une nuance de ton : est une erreur ce qui rendrait le diagramme *faux*, un avertissement ce qui le rendrait seulement *différent* de ce qu'on croyait demander. |
| `Strategie` | Le seul point délicat. Le défaut de `@UMLClass` renvoie au fichier `umlg.rules` — que `javac` ne sait pas trouver, puisqu'il compile des sources et ignore ce que la construction fera du résultat. Sans lui, tous les messages du genre « ce membre n'est pas affiché » deviennent indécidables, et le processeur *se tait* : un avertissement qui se trompe coûte plus cher que celui qui manque un cas, puisqu'il apprend à ne plus les lire. |

# 9 Ajouter une nature de lien : le mode d'emploi

C'est le scénario pour lequel l'architecture a été réorganisée. Supposons que vous vouliez ajouter `@UMLInheritance` (l'héritage). Voici la liste complète de ce qu'il faut faire :

1. **Déclarer l'annotation** : un nouveau fichier dans `annotation/`.
2. **Déclarer le lien dans le modèle** : un nouveau `record` dans `model/`, qui implémente `UmlRelation`, et une mention dans la clause `permits` de celle-ci.
3. **Écrire la lecture** : une classe dans `extraction/relation/` qui implémente `RelationExtractor` plus *une ligne* dans le catalogue `EXTRACTEURS` d'`EntityExtractor`.
4. **Écrire le dessin** : les `case` manquants dans `RelationRenderer` — le compilateur dit lesquels.

**Ce qui ne bouge pas.** Ni `UmlEntity`, ni le corps d'`EntityExtractor`, ni `PlantUmlSyntaxBuilder`, ni `PlantUmlWriter`. Auparavant, `UmlEntity` devait être modifié en cinq endroits, et `PlantUmlSyntaxBuilder` en deux.

**Ce que le compilateur garantit.** Si vous oubliez l'étape 4, le projet **ne compile pas**. Le compilateur pointe précisément les deux `switch` à compléter. C'est toute la raison d'être de l'interface *scellée* : auparavant, un lien non dessiné disparaissait simplement du diagramme, sans erreur, sans avertissement — le dessin paraissait juste un peu plus simple qu'il n'aurait dû.

**Pourquoi ouvert d'un côté et fermé de l'autre ?** L'asymétrie est délibérée, et repose sur ce que coûte l'oubli. Oublier un extracteur ne produit *rien* : la liste est explicite

et se relit. Oublier un dessin produit un diagramme *plausible* mais amputé d'une famille entière de traits — l'erreur la plus coûteuse, puisque rien ne la signale.

## 10 `src/test/` — les tests, en miroir

Les tests suivent exactement le découpage des sources : à chaque classe qui porte une responsabilité correspond un fichier de tests qui la vérifie, dans le même sous-dossier. Cela vaut mode d'emploi : pour comprendre ce que fait `NoiseFilter`, `NoiseFilterTest` est souvent plus instructif que le code lui-même, parce qu'il montre les cas limites qui ont motivé chaque ligne. Le fichier à ouvrir se déduit donc du nom de la classe, et il n'y a pas de catalogue à tenir à jour ici.

Deux familles échappent à ce miroir, et c'est délibéré.

**Les tests nommés d'après une fonctionnalité.** `PlantUmlEscapeTest`, `PlantUmlLisibiliteTest`, `AffichageDesMembresTest` portent le nom de ce qui est *rendu*, non de la classe qui l'implémente. C'est un axe de découpage tout aussi légitime, et il a un avantage propre : ces tests vérifient le document produit dans son ensemble, ce qu'aucun test d'une classe isolée ne pourrait faire.

**Les tests de bout en bout.** `DuCodeCompileAuPumlTest` écrit un petit projet, le *compile pour de bon*, pose un fichier de règles à côté et vérifie les `.puml` livrés. C'est le seul niveau où l'on voit la chaîne entière, et donc le seul où se voient les défauts qui vivent *entre* deux étapes — chacune irréprochable de son côté.

## 11 Récapitulatif en une page

| Dossier | En une phrase |
|---|---|
| `umlg/` | La porte d'entrée : une seule classe, `UMLGenerator`. |
| `annotation/` | Ce que l'utilisateur écrit. Ne valide rien, ne connaît aucun moteur de rendu. |
| `model/` | Ce que le diagramme dit. Immuable, se garde lui-même, ignore PlantUML. |
| `extraction/` | Comment on lit le code. Une responsabilité par fichier, parce que chacune grossit de son côté : `member/` remplit les boîtes, `relation/` tend les traits, `rules/` lit ce que le projet a réglé une fois pour toutes. |
| `rendering/` | Comment on dessine. Seul à connaître PlantUML, seul à encoder les textes. |
| `processing/` | Hors chaîne : la même vérification, mais dans le compilateur, donc des années-lumière plus tôt. |

**Si vous ne deviez retenir qu'une chose :** les liens de toutes natures vivent dans une seule liste, décrite par l'interface scellée `UmlRelation`. C'est elle qui fait qu'une annota-

tion nouvelle s'ajoute sans toucher au cœur, et qu'un lien oublié au dessin empêche la compilation au lieu de disparaître discrètement du diagramme.